Professional Vue.js

Completed source code for all labs (for checking your work) can be found at:

`https://github.com/watzthisco/pro-vue`

Version 1.0, March 2019
by Chris Minnick

www.watzthis.com

Table of Contents

Disclaimers and Copyright Statement

Disclaimer

WatzThis? takes care to ensure the accuracy and quality of this courseware and related courseware files. We cannot guarantee the accuracy of these materials. The courseware and related files are provided without any warranty whatsoever, including but not limited to implied warranties of merchantability or fitness for a particular purpose. Use of screenshots, product names and icons in the courseware are for editorial purposes only. No such use should be construed to imply sponsorship or endorsement of the courseware, nor any affiliation of such entity with WatzThis?.

Third-Party Information

This courseware contains links to third-party web sites that are not under our control and we are not responsible for the content of any linked sites. If you access a third-party web site mentioned in this courseware, then you do so at your own risk. We provide these links only as a convenience, and the inclusion of the link does not imply that we endorse or accept responsibility for the content on those third-party web sites. Information in this courseware may change without notice and does not represent a commitment on the part of the authors and publishers.

Copyright

Help us improve our courseware

Please send your comments and suggestions via email to info@watzthis.com

Credits

About the Author

Chris Minnick is a prolific published author, trainer, web developer and founder of WatzThis, Inc. Minnick has overseen the development of hundreds of web and mobile projects for customers from small businesses to some of the world's largest companies, including Microsoft, United Business Media, Penton Publishing, and Stanford University.

Since 2001, Minnick has trained thousands of Web and mobile developers. In addition to his in-person courses, Chris has written and produced online courses for Ed2Go.com, Skillshare, O'Reilly Media, and Pluralsight.

Minnick has authored and co-authored books and articles on a wide range of Internet-related topics including JavaScript, HTML, CSS, mobile apps, e-commerce, Web design, SEO, and security. His published books include JavaScript for Kids, Writing Computer Code, Coding with JavaScript For Dummies, Beginning HTML5 and CSS3 For Dummies, Webkit For Dummies, CIW eCommerce Certification Bible, and XHTML.

Setup Instructions

Course Requirements

To complete the labs in this course, you will need:

- A computer with MacOS, Windows, or Linux.
- Access to the Internet.
- A modern web browser.
- Ability to install software globally (or certain packages pre-installed as specified below).

Classroom Setup

These steps must be completed in advance if the students will not have administrative access to the computers in the classroom. Otherwise, these steps can be completed during the course as needed.

☐ 1. Install node.js on each student's computer.

   Go to **nodejs.org** and click the link to download the latest version from the LTS branch.

☐ 2. Install a code editor.

   We recommend the free Visual Studio Code editor, which you can download from https://code.visualstudio.com/

☐ 3. Make sure Google Chrome is installed.

☐ 4. Install git on each student's computer.

   If you're using MacOS, you already have git installed. If you're on Windows, git can be downloaded from **http://git-scm.com**. Select all the default options during installation.

Testing the Setup

☐ 1. Open a command prompt.
   - Use Terminal on MacOS (/Applications/Utilities/Terminal).
   - Use gitbash on Windows (installed with git).

☐ 2. Enter `cd` to navigate to the user's home directory (or change to a directory where student files should be created).

☐ 3. Enter the following:

   ```
   git clone https://github.com/watzthisco/pro-vue
   ```

   The lab solution files for the course will download into a new directory called `pro-vue`.

☐ 4. Enter `cd pro-vue/setup-test` to switch to the test project.

☐ 5. Enter `npm install`

   This step will take some time. If it fails, the likely problem is that your firewall is blocking ssh access to **github.com** and/or **registry.npmjs.org**.

☐ 6. When everything is done, enter `npm run serve`

☐ 7. If you get an error, delete the **node_modules** folder (by entering `rm -r node_modules`) and run `npm install` again, followed by `npm run serve`

Introduction and Git Repo Info

Most of the labs in this course build on the labs that came before. So, if you don't complete a lab or can't get a certain lab to work, it's possible that you can get stuck and won't be able to move forward until the error is corrected.

To help you check your work and to make it possible to come into the class at any point, the git repository for this course contains finished versions of every lab.

The url for the course repository is:
`https://github.com/watzthisco/pro-vue`

You can find the finished code for each lab inside the **solutions** directory.

Lab 01 - Installing and Configuring VS Code

Visual Studio Code is an Integrated Development Environments for JavaScript. You can use any code editor or IDE you like, but Visual Studio Code is open source and has good support for modern frameworks like Vue.js and Node.js, as well as built-in integrations with the tools we'll be using in this course and plugins for doing anything else you'll need to do.

None of the labs in this course (other than this one) will depend on any particular IDE; so, if you prefer another editor, feel free to use it and to adapt instructions in this lab to your own editor.

Part 1 - Installing Visual Studio Code

- ☐ 1. Go to https://code.visualstudio.com/ .
- ☐ 2. Click the **download** link.

When the download completes, launch the installer and follow the prompts to install Visual Studio Code.

Part 2: Installing Extensions

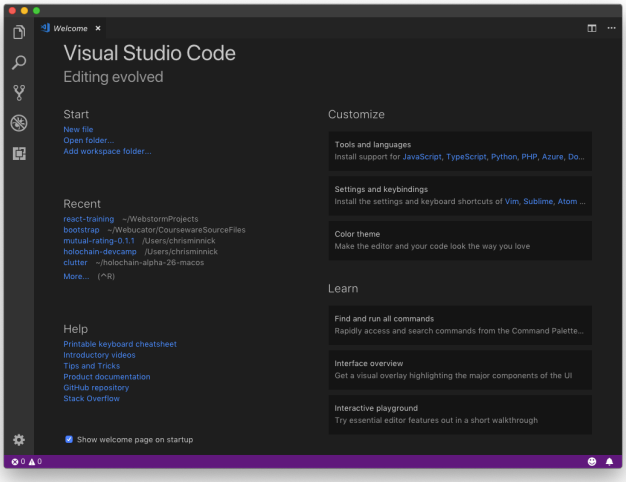
VS Code has hundreds of available extensions. To make working with Vue.js and related JavaScript tools easier, we'll install a few of them now.

- ☐ 1. Click the Extensions icon on the left sidebar. It looks like this:

- ☐ 2. Search for the 'open in browser' extension, by TechER. This extension will allow you to click on files in your project and quickly open them in a web browser to preview them. When you find it, click the Install icon next to it.
- ☐ 3. Search for and install the Vetur extension, by Pine Wu. This extension will give you Vue.js syntax highlighting, formatting, linting, and debugging capabilities.

Part 3: Creating a New Project

- ☐ 1. When you start VS Code, you'll see the splash screen:



☐ 2. Select File > Open or click the Open Folder link under Start on the default start screen.

☐ 3. Select the folder where you want to create your project (such as in My Documents) and click New Folder to create a new folder. Name the new folder **react-training**.

☐ 4. Select the new folder and click **Open**.

Lab 02: Get Started with vue-cli

☐ 1. Start a new project in VS Code (if you didn't already do this in the previous lesson).

☐ 2. Open the terminal `(ctrl - `)` and install or update vue-cli

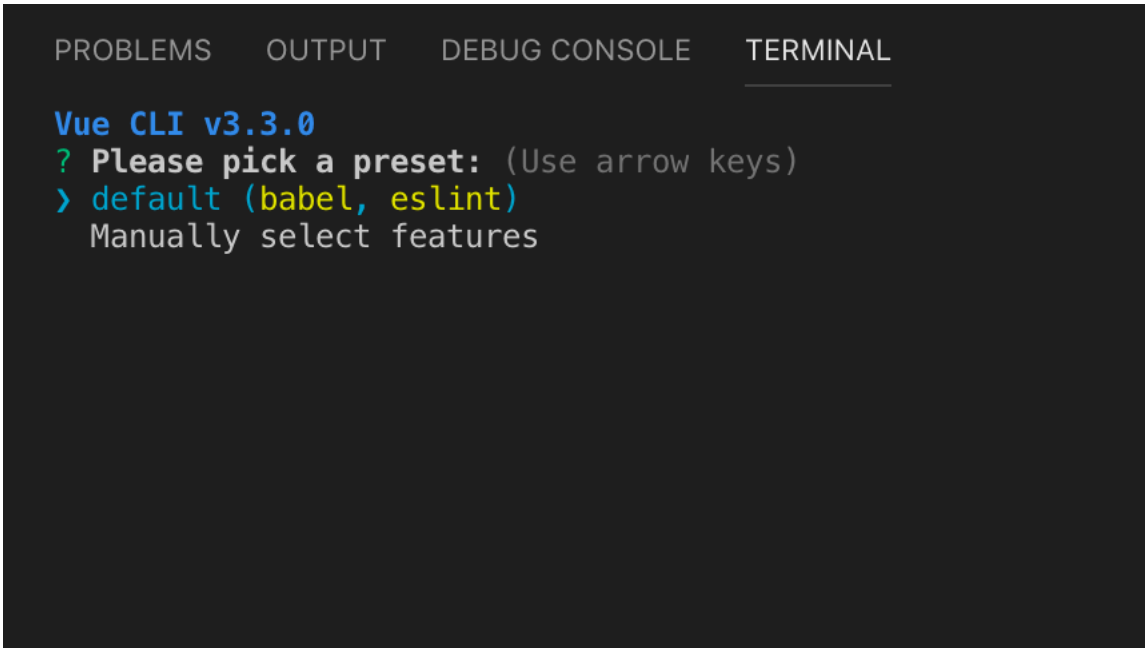
```
npm install -g @vue/cli
```

☐ 3. Enter the following to create your first app:

```
vue create conduit
```

If this produces an error, you most likely need to upgrade the version of node and npm on your computer (see the setup instructions).

☐ 4. Select the default preset (babel, eslint).



☐ 5. Wait while your new project is created. It may take a few minutes.

☐ 6. Go into the new directory.

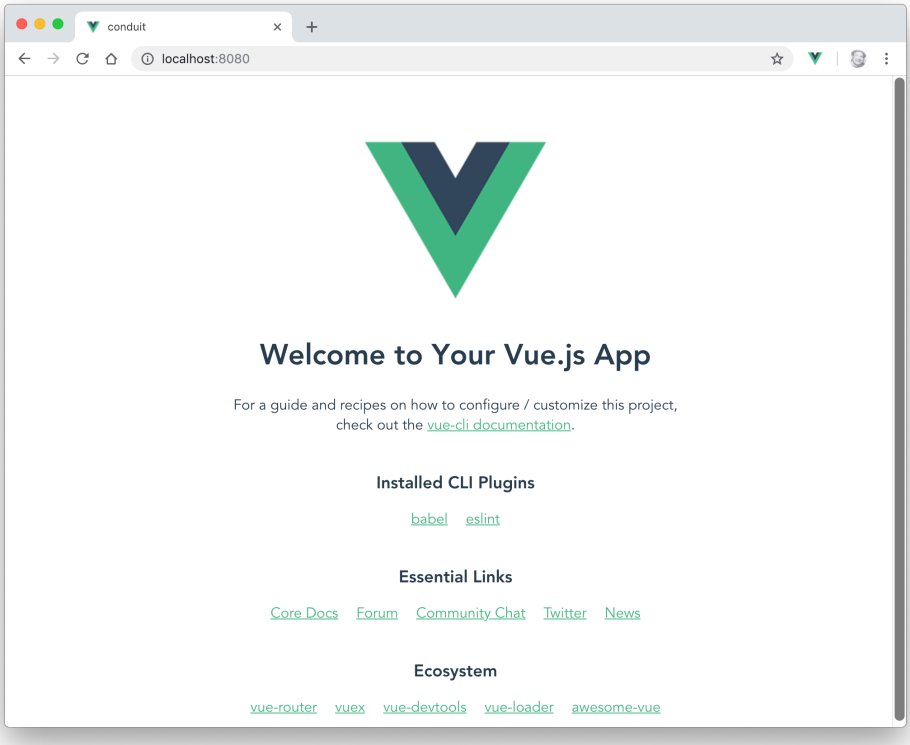
```
cd conduit
```

☐ 7. Test that everything was installed and works.

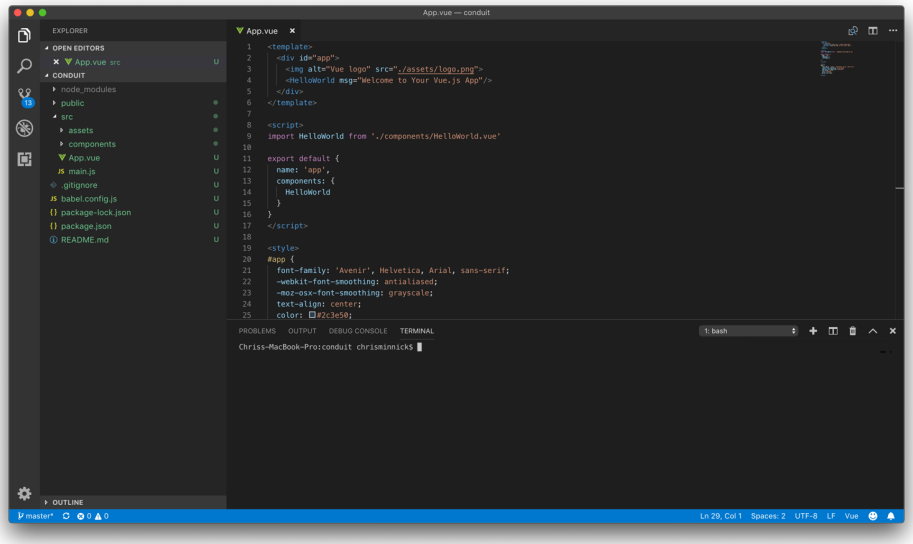
```
npm run serve
```

☐ 8. When the server is fully started, you'll see a message with the localhost and network URLs where you can access your app. Go to the localhost URL in a web browser (localhost:8080 by default).

If the app was successfully created, you should see the following page.



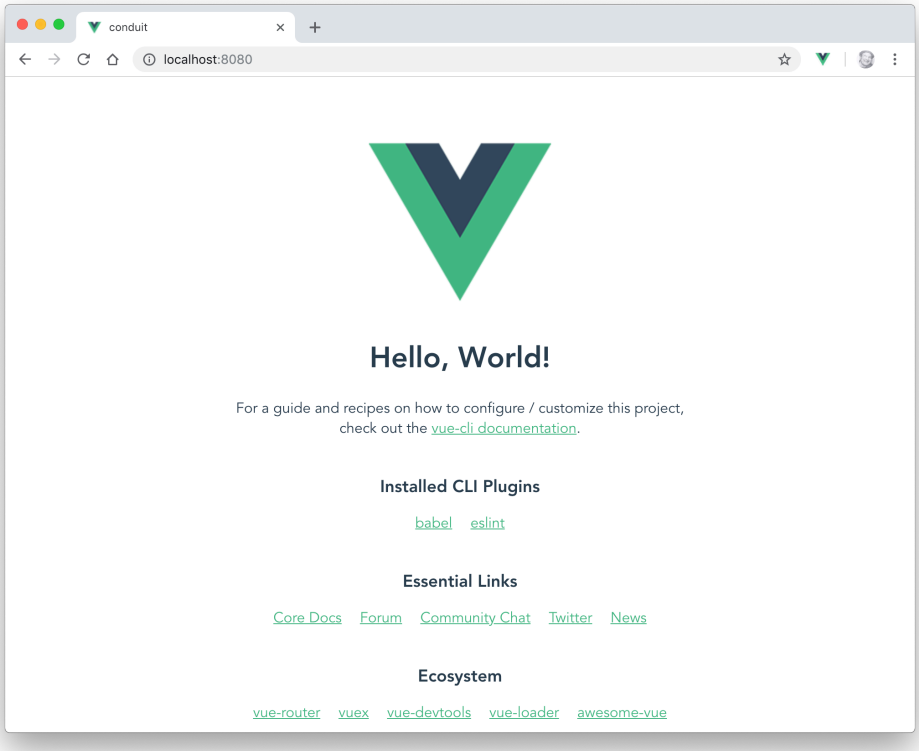
☐ 9. In your code editor, Find **App.vue** inside the **src** directory and open it for editing.



☐ 10. Inside the `<template>` section at the beginning of the file, change the value of the `msg` attribute to any message you want.

☐ 11. Return to your web browser and notice that the text has been automatically refreshed.

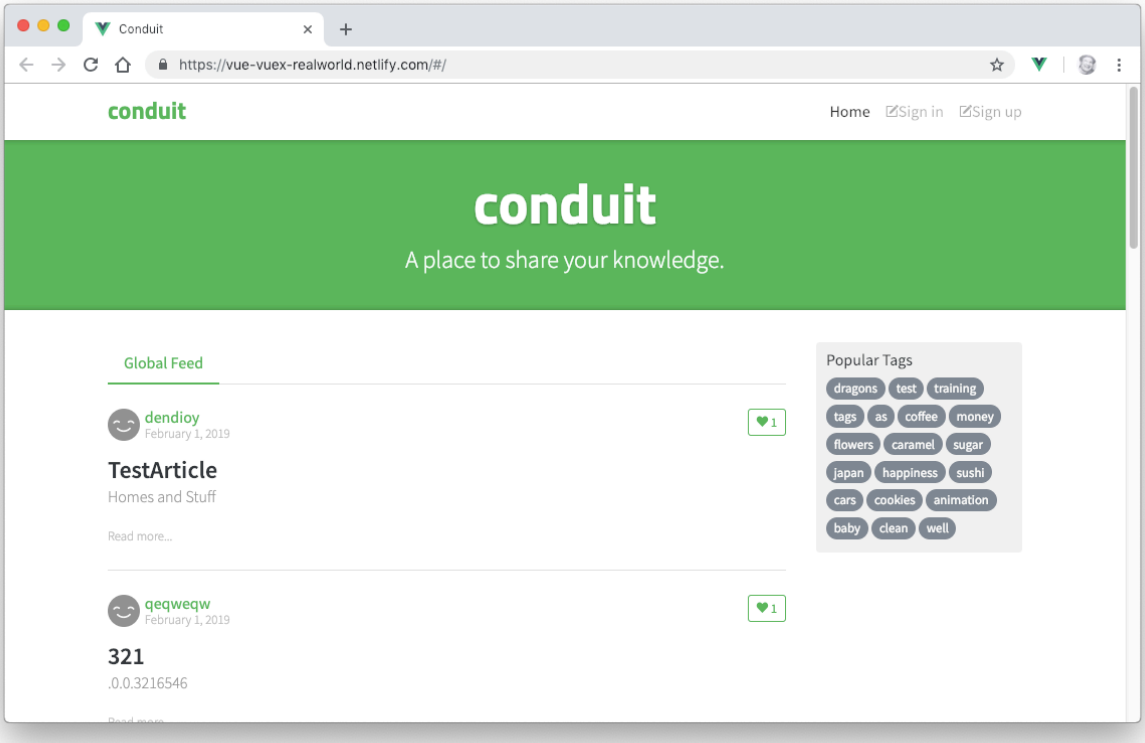
If it's doesn't refresh, return to your Terminal emulator and enter **npm run serve**.



Introducing Our Real-World Project: Conduit

In this class, we'll be building a real-world application based on **Conduit**, an open source Medium.com clone. This application will make use of a wide variety of Vue.js features and related technologies, including components, authentication, routing, state management, a RESTful API and more.

Before continuing to the next lab, visit an instance of Conduit at https://conduit.watzthis.com.



Lab 03: Your First Component

Vue.js components let you divide your user interface into independent and reusable pieces. The simplest components simply output some piece of HTML, given some input.

In this lab, you'll create a component to hold the contents of the page header.

☐ 1. Create a new file named **Header.vue** in the **src/components** directory

This component will hold the navigation and logo for our app. At this point, this component will just render static HTML. We'll be using the Bootstrap CSS framework to style the navigation as responsive horizontal navbar.

☐ 2. Inside Header.vue create a template element and a script element:

```
<template>
</template>
<script>
</script>
```

☐ 3. Inside the script element, export the component and give it the name of Header:

```
export default {
  name: 'Header'
}
```

☐ 4. Inside the `template` element, create a `nav` element containing a `div` element, and give the `div` element a `class` of `container`. The `container` class will cause Bootstrap to style the contents of the `nav` element as a responsive container.

```
<template>
<nav>
  <div class="container">
  </div>
</nav>
</template>
```

☐ 5. Give the nav element the Bootstrap `navbar` class.

```
<template>
<nav class="navbar">
  <div class="container">


  </div>
</nav>
</template>
```

☐ 6. Make a logo link in the container by creating a link and giving it the navbar-brand class.

```
<template>
<nav class="navbar">
  <div class="container">
    <a class="navbar-brand">Logo</a>
  </div>
</nav>
</template>
```

☐ 7. Below the logo link create a list using a ul element and several li elements.

```
<div class="container">
  <a class="navbar-brand">Logo</a>
  <ul>
    <li>Home</li>
    <li>Sign In</li>
    <li>Sign Up</li>
  </ul>
</div>
```

☐ 8. Add Bootstrap's `nav` and `navbar-nav` classes to the `ul` to format it as a navbar, and add the `flex-row` class to make it a horizontal navbar.

```
<ul class="nav navbar-nav flex-row">
  <li>Home</li>
  <li>Sign In</li>
  <li>Sign Up</li>
</ul>
```

☐ 9. Add `nav-item` classes to each of the `li` elements to turn it into a navbar item, and add `p-2` to add some padding around each item.

```
<ul class="nav navbar-nav flex-row">
  <li class="nav-item p-2">Home</li>
  <li class="nav-item p-2">Sign In</li>
  <li class="nav-item p-2">Sign Up</li>
</ul>
```

☐ 10. Turn the text in each `nav-item` into a link and add the `nav-link` class.

```
<ul class="nav navbar-nav flex-row">
  <li class="nav-item p-2">
```

```
      <a class="nav-link">Home</a>
    </li>
    <li class="nav-item p-2">
      <a class="nav-link">Sign In</a>
    </li>
    <li class="nav-item p-2">
      <a class="nav-link">Sign Up</a>
    </li>
  </ul>
```

☐ 11. When it's done, **Header.vue** should look like this:

```
<template>
<nav class="navbar">
   <div class="container">
      <a href="/" class="navbar-brand">Conduit</a>
      <ul class="nav navbar-nav flex-row">
        <li class="nav-item p-2">
          <a class="nav-link">Home</a>
        </li>
        <li class="nav-item p-2">
          <a class="nav-link">Sign In</a>
        </li>
        <li class="nav-item p-2">
          <a class="nav-link">Sign Up</a>
        </li>
      </ul>
    </div>
</nav>
</template>

<script>
export default {
  name: 'Header'
}
</script>
```

☐ 12. Add the following to **App.vue** (above the other import statement)

```
import Header from './components/Header';
```

☐ 13. Add **Header** to the components property inside **App.vue**

```
export default {
  name: 'app',
  components: {
    Header,
    HelloWorld
  }
}
```

☐ 14. Add the following inside the **<template>** in **App.vue** (above the Vue logo).

```
<Header />
```

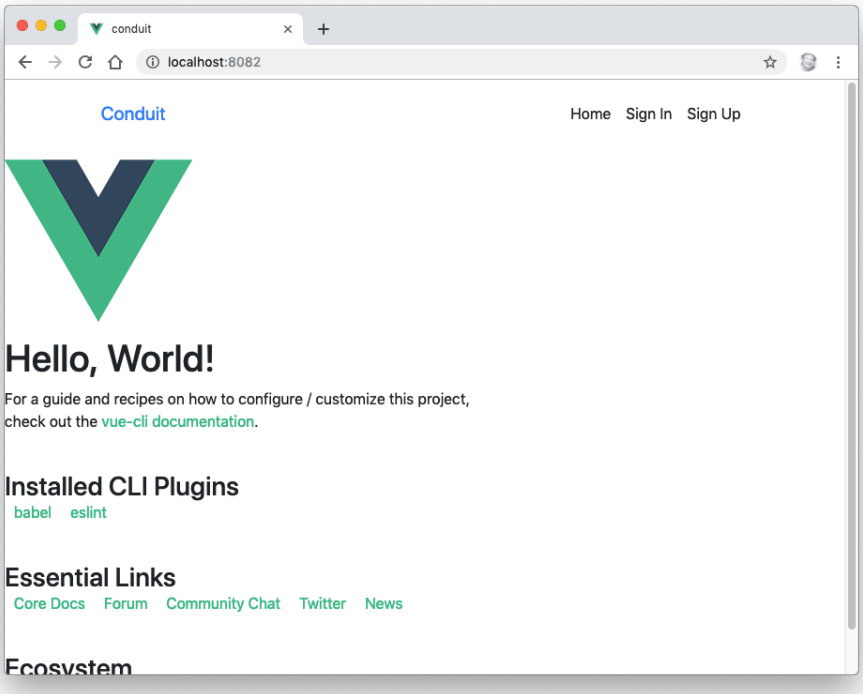
☐ 15. Delete the contents of the `<style>` element in **App.vue**.

☐ 16. Open **public/index.html** in Visual Studio Code.

☐ 17. In your browser, go to https://www.bootstrapcdn.com. Scroll down the page and click the arrow to the right of the Complete CSS URL. Copy the HTML link element that appears and paste it into the `head` element in **index.html**.

☐ 18. Expand the Complete JavaScript and copy the `script` tag into **index.html**, right above the `</body>` tag.

☐ 19. Start the app (if it's not already running) with `npm run serve`, and view it in your browser.



Lab 04: Create More Components

In this lab, you'll create static versions of the main body and the footer for your Vue application.

☐ 1. Update the template in App.vue to contain the three main components that will make up the application's Home page: Header, Home, and Footer. Remove HelloWorld from the template, the export, and from the imports section in App.vue.

```
<template>
<div id="app">
  <Header />
  <Home />
  <Footer />
</div>
</template>
```

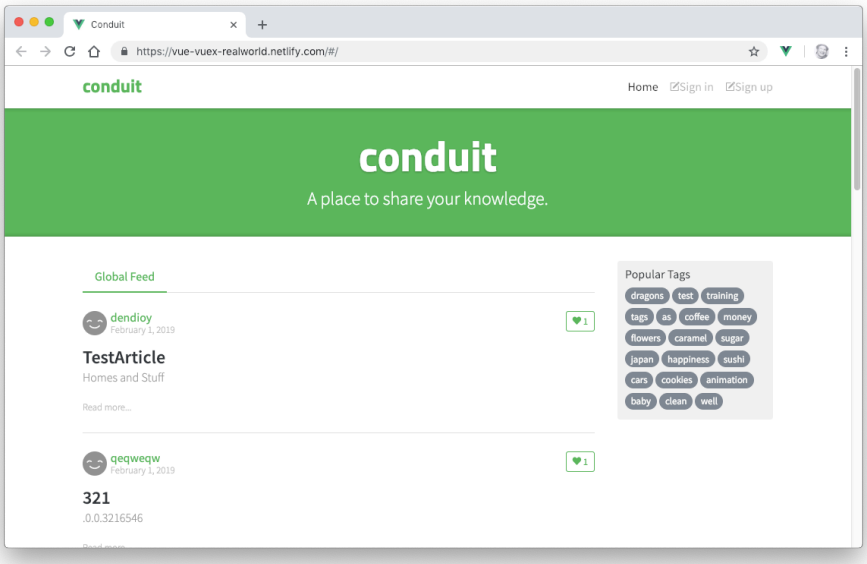
☐ 2. Make the new .vue files for the Home and Footer components (inside **src/components**) and create template and script sections for each. Write the export statements for each component.

☐ 3. Put some placeholder text into the new components (for example: <h1>Home</h1> and <h1>Footer</h1>.

☐ 4. Import Footer and Home into App.vue and add them to the components property in the export as well.

☐ 5. Run npm run serve and preview your app so far in a browser.

☐ 6. Using the following screenshot of the finished Conduit application, write the HTML in the Home.vue component to render a static version of everything underneath the top navigation (without worrying about style yet). The next page contains the finished code, but try to implement it on your own before looking.

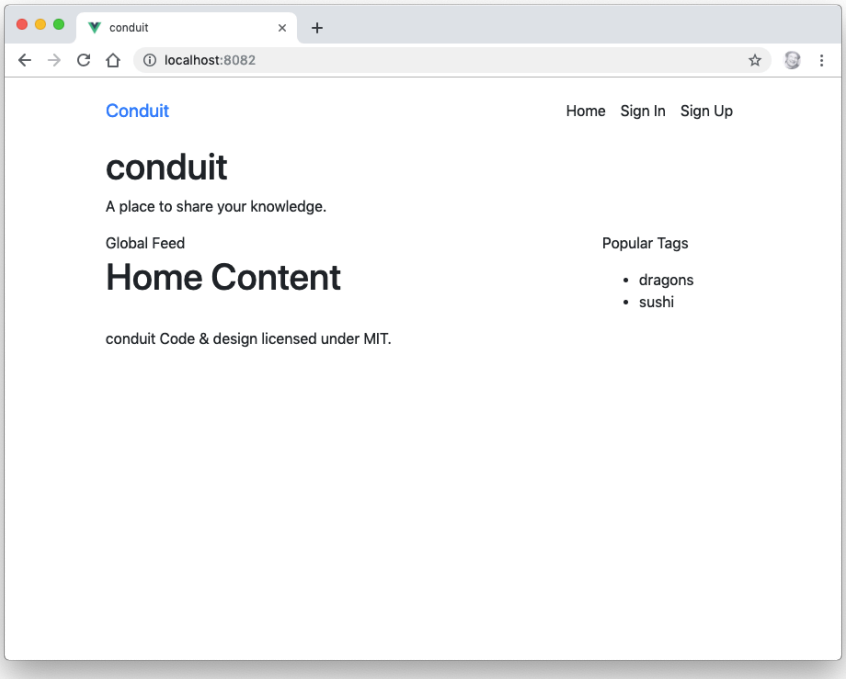


```
<template>
  <div class="home-page">
    <div class="banner">
      <div class="container">
        <h1 class="logo-font">conduit</h1>
        <p>A place to share your knowledge.</p>
      </div>
    </div>
    <div class="container page">
      <div class="row">
        <div class="col-md-9">
          <div class="feed-toggle">
            <ul class="nav nav-pills outline-active">
              <li class="nav-item">Global Feed</li>
            </ul>
          </div>
        <p>Home Content Placeholder</p>
        </div>
        <div class="col-md-3">
          <div class="sidebar">
            <p>Popular Tags</p>
            <div class="tag-list">
                <ul class="tag-list">
                    <li class="tag-default tag-pill tag-outline">
                       dragons
                    </li>
                    <li class="tag-default tag-pill tag-outline">
                       sushi
                    </li>
                </ul>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</template>
```

☐ 7. Write the HTML for the static version of the Footer component.

Here's what the final rendered footer should be:

```
<footer>
  <div class="container">
    conduit
    <span class="attribution">
        Code &amp;amp; design licensed under MIT.
    </span>
  </div>
</footer>
```


conduit
localhost:8082
Conduit
Home
Sign In
Sign Up
conduit
A place to share your knowledge.
Global Feed
Home Content
Popular Tags
dragons
sushi
conduit Code & design licensed under MIT.

Lab 05: Testing Vue

In this lab, you'll create simple unit tests for each of the components you've made so far.

- ☐ 1. Make a new directory (at the same level as the **src** directory) and name it **tests**.
- ☐ 2. Make a subdirectory of your **tests** directory and name it **unit**.
- ☐ 3. Make a subdirectory of your of **unit** directory and name it **components**.
- ☐ 4. Make a new file in this components unit test directory named **Header.spec.js**.
- ☐ 5. Import **mount** from the vue test utilities into this file.

```
import { mount } from "@vue/test-utils";
```

- ☐ 6. Import the Header component.

```
import Header from "../../../src/components/Header.vue";
```

- ☐ 7. Write a function to mount the component.

```
const createWrapper = () => {
  return mount(Header);
};
```

- ☐ 8. Write a describe function to describe your test suite.

```
describe("Header", () => {
})
```

- ☐ 9. Inside the describe function, create a test.

```
  it("should render without a problem", () => {
      const wrapper = createWrapper();
      expect(wrapper.exists()).toBe(true)
  });
```

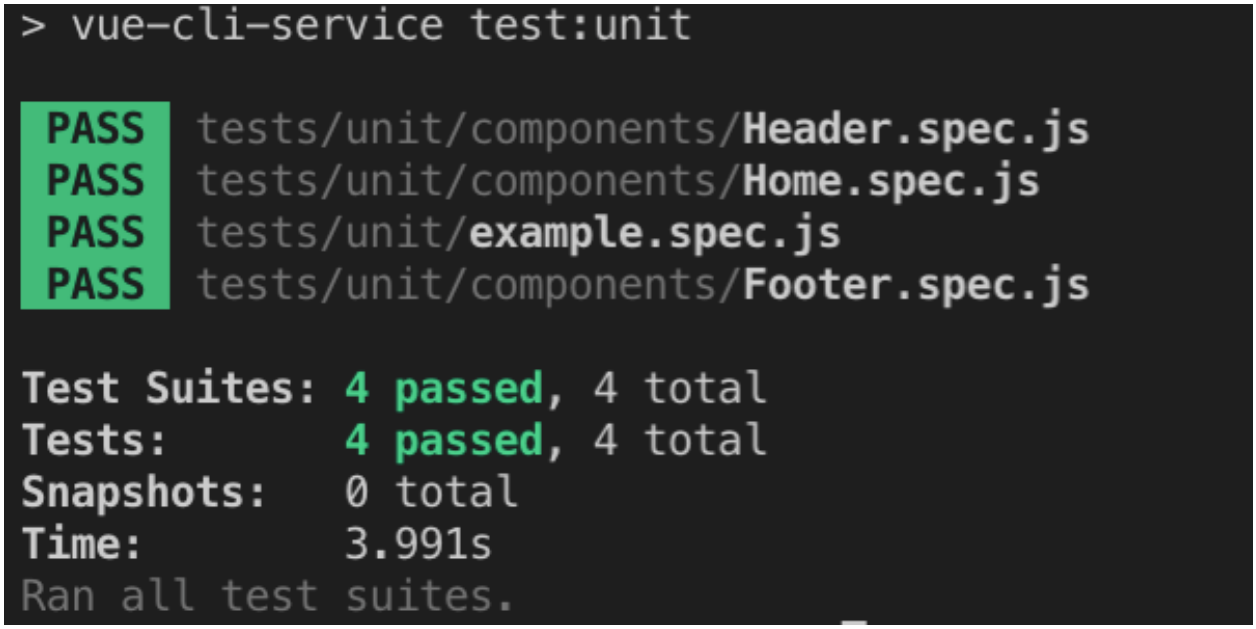
- ☐ 10. Install the vue-cli Jest plugin

```
vue add @vue/cli-plugin-unit-jest
```

- ☐ 11. Run `npm run test:unit` to check whether it works.
- ☐ 12. Make copies of this spec for the **Home** component and the **Footer** component
- ☐ 13. Modify the contents of the new files to test the new components.
- ☐ 14. Run your tests by entering the following in the command line

```
npm run test:unit
```

- ☐ 15. Make sure that all the tests pass.


> vue-cli-service test:unit
PASS tests/unit/components/Header.spec.js
PASS tests/unit/components/Home.spec.js
PASS tests/unit/example.spec.js
PASS tests/unit/components/Footer.spec.js
Test Suites: 4 passed, 4 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 3.991s
Ran all test suites.

Lab 06 - Getting Started with Node.js and npm

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. It can be used to create server-side programs with JavaScript as well as for automating development tasks. In this course, we will be using it for the latter purpose.

Part 1: Getting to Know Node.js

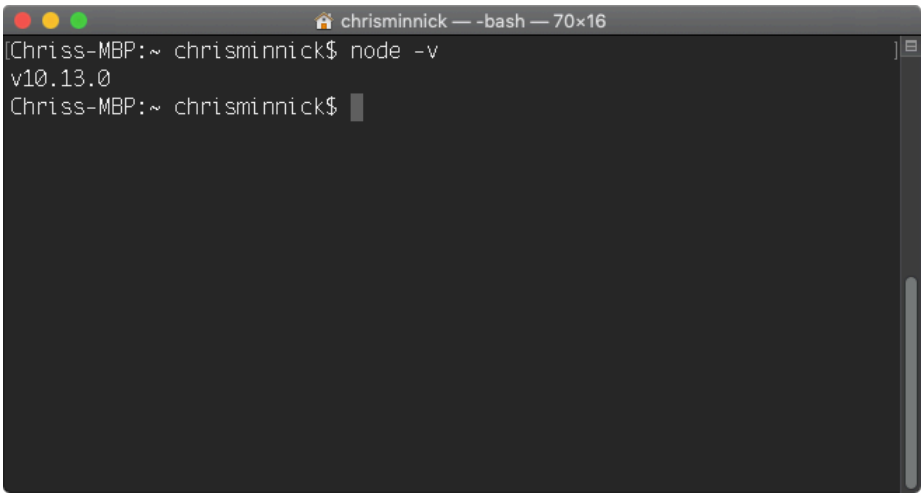
In this part, you will learn the basics of using Node.js.

☐ 1. Open a terminal window (such as the one built into VS Code) and change to the labs/lab06 directory.

Note: You can use the cd command (MacOS and Windows) to change directories. To go up a directory use `cd ../`

To go into a directory, enter `cd` followed by the name of the directory. You can list the contents of a directory by using `ls` (on MacOS) or `dir` (on Windows).

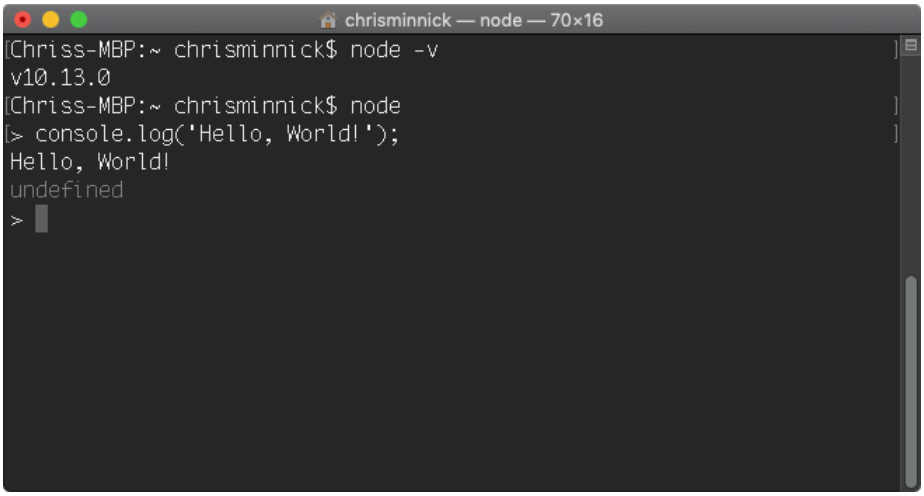
☐ 2. To check whether Node.js is property installed, enter `node -v`
You should see something like the following:



☐ 3. Enter `node` to open the interactive shell, REPL.

Note: You can enter any JavaScript statement into REPL and you have access to all the Node.js modules.

☐ 4. Enter `console.log('Hello, World!');` into the shell.



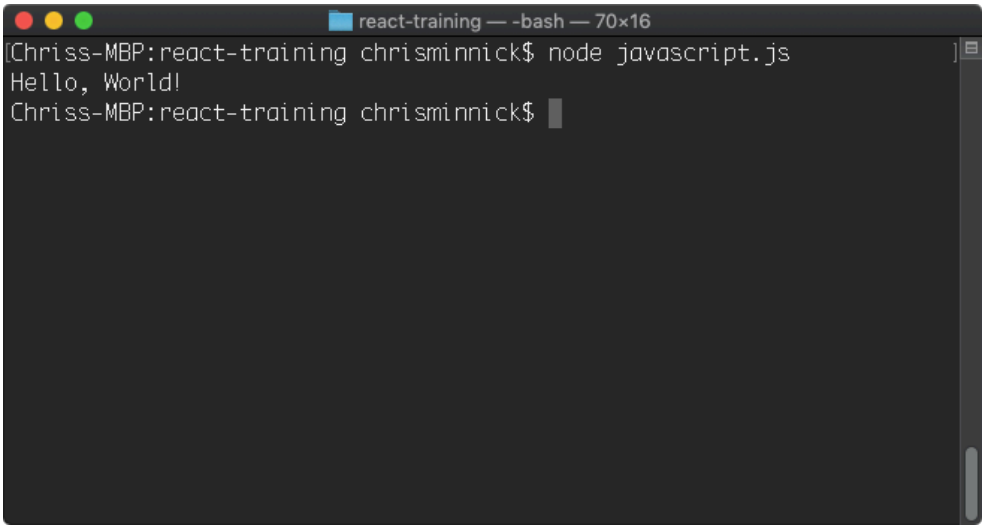
Note: Every JavaScript statement has a return value. The default return value is `undefined`. So, if you execute a command that doesn't have any other return value, as in this case, node outputs `undefined` after the results of running the statement.

You will not normally work with node from the interactive shell. The other way to execute code with node is to write your JavaScript into a file and execute that file.

- ☐ 5. Exit out of REPL by pressing ctrl-D (Mac or Linux) or ctrl-Z (Windows) or by pressing ctrl-C twice.
- ☐ 6. Create a text file using your code editor and enter the following code:

```
console.log('Hello, World!');
```

- ☐ 7. Save the file as **javascript.js**
- ☐ 8. In your terminal, navigate to the directory where you saved javascript.js.
- ☐ 9. Once you've located javascript.js, enter `node javascript.js` to run it.



Part 3: Using npm

The node package manager (npm) is the tool for installing and managing node modules created by the node community. In this part, you will learn about the basic npm commands.

☐ 1. Enter `npm -v` to find out what version of npm is installed on your computer.
☐ 2. Enter `npm install npm -g`

This command will install the latest version of npm.

Note: If the installation of npm fails on MacOSX, you may need to preface it with sudo in order to install as the super user.

☐ 3. Enter `npm -v` to see what version of npm is now installed.
☐ 4. Enter `npm ls -g`

This command will list all the packages that are installed on your computer currently. Use it without the `-g` to see only packages installed into your current project.

☐ 5. Enter `npm help ls`

The help command will show you documentation for a package. On Windows, it may open in a browser. On MacOS, the help will display in the Terminal.

☐ 6. If the help file displayed in the console window, type `q` to exit the help system.
☐ 7. Enter `npm update` or `npm update -g`

`npm update` will search the npm registry for newer versions of installed packages and install them along with their dependencies.

These are all the basic commands you need to know to get started with npm. In future labs, we will be using npm extensively to install and manage packages and to run tasks.

Lab 07 - Initialize npm

In this lab, you will initialize npm for your project and learn about the package.json file.

☐ 1. In your terminal, change to the **labs/lab07** directory and enter:

```
npm init
```

You will be asked some questions to configure npm for your project. The default values will be shown in parentheses after the question. Press Enter or Return to accept each of these default values. Once you have gone through all the questions, you will see that a new file, package.json, has been created in the root of your project.

Note: When using git bash shell on Windows, the configuration script may hang after the last question. When this happens, press Ctrl-C. Everything has run correction and the package.json file has been created, but it just doesn't exit correctly.

☐ 2. Open **package.json** in your code editor.

The package.json file configures npm. When you want to install your project in a new directory, you will enter `npm install` and it will follow instructions in this file to do the job.

☐ 3. Enter `npm install` in the console.

There's nothing for npm to do at this point, since you don't have any modules installed or instructions inside **package.json**, however a new file named **package-lock.json** will be created in your project.

Part 2: How to Write and Run npm Scripts

In this part, you will learn how to create npm scripts and run them. Npm scripts can be used to automate many of the tasks involved in front-end development, such as testing, building, and deployment.

☐ 1. Open package.json in your code editor if it's not already open.

Npm's default package.json file contains a scripts object. If you didn't specify a test script when you ran npm init, a default test method was created for you inside the scripts object.

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

☐ 2. Open your command prompt and enter `npm run`.

Npm's run command can be used to run methods inside the scripts object. If you use npm run without any arguments, it will return a list of the available scripts. In this case, you should get the following:

```
Lifecycle scripts included in lab-files:
  test
    echo "Error: no test specified" && exit 1
```

☐ 3. Enter `npm run test`

The test script will run and output the message saying that no test is specified, and then it will exit with an error.

We'll specify a test in a future lab. For now, we're going to create a simple build script, which will run the test script and then exit with a message.

☐ 4. Change `exit 1` in the test script to `exit 0` and run the test script again.

`Exit 0` tells node to exit normally without throwing an error.

☐ 5. Add another property to the scripts object, named build. For now, the build script will just print out a message.

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "build": "echo \"BUILD OK\" && exit 0"
},
```

☐ 6. Create a prebuild script. Each script that you create automatically has a pre- and post- script that you can override with your own code. The names of these scripts are the name of your method, with either pre or post prepended. We'll use the prebuild script to run the test script, and we'll change the test script to output a success message for now. **Make sure to change the exit status to 0 to indicate success.**

```
"scripts": {
  "test": "echo \"All tests passed\" && exit 0",
  "build": "echo \"BUILD OK\" && exit 0",


  "prebuild": "npm run test"
},
```

☐ 7. Return to the command line, and type **`npm run build`** at the command line to test your new (very simple) automated build.

Lab 08: Making a Web Server

In this lab, you'll use Node.js and the `http` module to create a simple web server that listens for connections and responds with a simple Hello World message.

☐ 1. Open your code editor and create a file named **server.js** inside the **labs/lab08** folder.

☐ 2. Enter the following code inside of server.js

```
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer(function(req, res) {
    res.statusCode = 200;
    res.setHeader('Content-Type', 'text/plain');
    res.end('Hello World\n');
});

server.listen(port, hostname, function() {
    console.log(`Server running at
http://${hostname}:${port}/`);
});
```

Note: The console.log statement in this program uses a new (ES6) JavaScript feature called template literals to combine dynamic values with static text. The characters surrounding the string starting with "`Server running at`" are backticks (in the upper left of the keyboard), not single quotes.

☐ 3. Save the **server.js** file.

☐ 4. In your terminal or command line, navigate to the **labs/lab08** folder and run the program by typing:

`node server.js`

If everything works correctly, you should see a message that the server is running.

☐ 5. In your web browser, go to the address shown in your console window, which should be **http://127.0.0.1:3000**

The server will return a message that will display in your browser.

☐ 6. Modify the script to return a different message or a full html page.

Lab 09 - Automate Linting

Linting is a way to perform static code analysis on your files. Static code analysis will look at the syntax (and the style, in some cases) of your JavaScript and alert you if there are problems.

In this lab, you will install ESLint, use it to check a JavaScript file.

Note: The configuration script for eslint currently works better in the Windows cmd.exe than in Git Bash. For this reason, you may want to use cmd.exe for the following steps.

☐ 7. If your terminal program isn't already open, open it and go to labs/lab09.
☐ 8. Type **npm install eslint --save-dev** to install ESLint.
☐ 9. Run **`./node_modules/.bin/eslint --init`** to set up the configuration file (use .\node_modules\.bin\eslint --init if you're working in cmd.exe).
☐ 10. Select **To check syntax, find problems, and enforce code style** as the answer to the first question.
☐ 11. Answer the questions as follows unless you have a good reason to answer differently. Don't worry if you make a mistake, we'll set all of the options correctly in the config file.
- What type of modules does your project use? **JavaScript modules**
- Which framework does your project use? **Vue.js**
- Where does your code run? **Browser**
- How would you like to define a style for your project? **Answer questions about your style**
- What format do you want your config file to be in? **JavaScript**
- What style of indentation do you use? (Your choice)
- What quotes do you use for strings? (Your choice. I recommend single)
- What line endings do you use? (Select Windows if you use **Windows**. Otherwise, select **Unix**)
- Do you require semicolons? **Y**
- What format do you want your config file to be in? **JavaScript**

Note: The init script may hang after the last question when using Git Bash shell. Use **Ctrl+C** to exit after the message appears that says "Successfully created .eslintrc.js".

☐ 12. Make a file named **.eslintignore** in the root of your directory.
☐ 13. Add the following text to **.eslintignore**

```
node_modules/*
```

☐ 14. Create a new script in package.json called **`lint`**, as follows:

```
"lint": "eslint . --ext .js",
```

I recommend putting it before the "test" script.

☐ 15. Run **npm run lint**.
☐ 16. Fix the errors reported by ESLint, or adjust the `.eslintrc.js` config file (which was created in the root of the project earlier in these steps) to fit your desired coding style.
☐ 17. If you're on Windows, you may need to change the line break style to **`"windows"`**. You should also add a **`"no-console"`** option with a value of

"**warn**" to override the default value of "error", since we'll be using console.log in upcoming labs.

Here's an example of the .eslintrc.js file.
.eslintrc.js

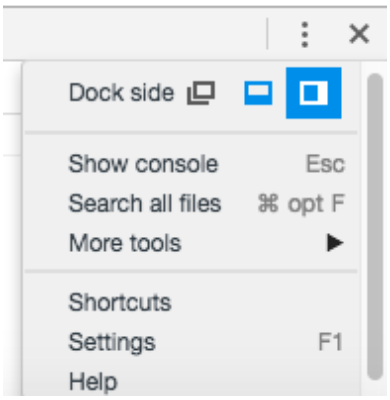
```
module.exports = {
    'env': {
        'browser': true,
        'es6': true
    },
    'extends': [
        'eslint:recommended',
        'plugin:vue/essential'
    ],
    'globals': {
        'Atomics': 'readonly',
        'SharedArrayBuffer': 'readonly'
    },
    'parserOptions': {
        'ecmaVersion': 2018,
        'sourceType': 'module'
    },
    'plugins': [
        'vue'
    ],
    'rules': {
        'indent': [
            'error',
            4
        ],
        'linebreak-style': [
            'error',
            'unix'
        ],
        'quotes': [
            'error',
            'single'
        ],
        'semi': [
            'error',
            'always'
        ],
        'no-console': [
            "warn"
        ]
    }
};
```

☐ 18. Make the lint script run prior to the test script in the prebuild script.

Lab 10: Manual In-Browser Testing and Debugging

In this lab, you will get acquainted with Google Chrome's Web Developer tools for inspecting and debugging your front-end code.

- ☐ 1. In your terminal, navigate to the **conduit** folder you were working with in labs 01-05.
- ☐ 2. Enter `npm run serve` to start the vue-cli development web server.
- ☐ 3. Open your Chrome browser and go to https://chrome.google.com/webstore
- ☐ 4. Search for the **Vue.js DevTools** extension and install it.
- ☐ 5. Restart your browser, then go to **http://localhost:8080**.
- ☐ 6. Press **Command**-**Option**-**I** (on MacOS) or **Ctrl**-**Shift**-**I** (Windows) to open the Developer Tools.
- ☐ 7. Dock the Developer Tools to the right side by clicking the **Customize** button on the right side of the Developer Tools toolbar and selecting **Dock Right**.



- ☐ 8. Click **Elements**. The current HTML and CSS of your document (as it exists in the DOM) will appear.
- ☐ 9. Expand the element and click on the Vue logo.
- ☐ 10. In the styles pane, add `border: 10px solid red;` to the `element.style` object.
- ☐ 11. Right-click the `h1` element containing "Vue.js is fun" and select `hide element`.
- ☐ 12. Click the **Console** tab to open the JavaScript console.

> You can also open the JavaScript console at any time by pressing **Ctrl**-**Shift**-**J** (Windows) or **Command**-**Option**-**J** (Mac).

- ☐ 13. Enter the following into the console, followed by Return (or Enter):

```
document.body.innerHTML = '<h1>Here is some new text!</h1>';
```

The content of the document's `body` element will change to the HTML you just entered.

- ☐ 14. Click the **Sources** tab.

The JavaScript debugger will open.

- ☐ 15. Click on **app.js** and click the line number next to any statement to set a breakpoint.
- ☐ 16. Refresh the page.

Execution of the script will halt prior to the statement running. Clearly, this is a very basic example that doesn't show us much about how the debugger works. But, examine the different options available and hover over the different buttons to find out what they do.

- ☐ 17. Visit *https://developers.google.com/web/tools/chrome-devtools/debug/breakpoints/?hl=en* to learn more about the Sources Panel and working with breakpoints.

Lab 11: Static Version

The first step in creating a Vue UI is to create a static version. In this lab, you'll continue creating the components that will make up the homepage of the Conduit example application, and you'll learn how to pass data from a parent component to child component and to start using directives to make templates dynamic.

The next thing we'll do is to create the list of articles in the center part of the design.

☐ 1. Make a new component named `GlobalFeed`. This will contain the list of articles that will display for logged-out users. Use the following template:

```
<template>
  <div class="home-global"><ArticleList type="all" /></div>
</template>
```

and export the component using the following `<script>` block:

```
<script>
export default {
  name: 'GlobalFeed'
}
</script>
```

☐ 2. Import a new component named `ArticleList` into the `GlobalFeed` component, and add it to the `components` object:

```
<script>
import ArticleList from './ArticleList.vue';

export default {
      name: 'GlobalFeed',
      components: {
            ArticleList
      }
}
</script>
```

☐ 3. Make a component named `ArticleList`. This will display a list of article previews. Use this template:

```
  <template>
  <div>
      <div v-if="articles.length === 0"
           class="article-preview">
        No articles are here... yet.
```

```
      </div>
      <ArticlePreview
        v-for="(article, index) in articles"
        :article="article"
        :key="article.title + index"
      />
    </div>
  </div>
</template>
```

This component makes use of the `v-if` directive to check whether there are any articles to display. It then uses the `v-for` directive to output an `ArticlePreview` component for each article.

We're also passing two props into the ArticlePreview component: `article`, and `key`.

This template tells us that we'll need to include a few more things in the script block for this component.

☐ 4. Start by importing a component (which we'll create in a moment) named `ArticlePreview`:

```
import ArticlePreview from './ArticlePreview.vue';
```

☐ 5. Next, copy the `articles.json` from the **labs/lab12** folder and put it into a subfolder of **src** named **json**. This file contains our article data (until start using a web API in a future lesson).

☐ 6. Import **articles.json** into `ArticleList`.

```
import articles from '../json/articles.json';
```

☐ 7. Next, create the export statement, with the components object (making sure to include `ArticlePreview`)

```
export default {
    name: 'ArticleList',
    components: {
    ArticlePreview
    }
}
</script>
```

☐ 8. Finally, create the `data` function, which will make this imported json data available to instances of `ArticleList`.

```
export default {
    name: 'ArticleList',
```

```
    components: {
    ArticlePreview
  },
  data(){
        return{
            articles: articles
        }
    }
}
</script>
```

☐ 9. Make a component named `ArticlePreview`. This will determine how each preview looks and will eventually contain a link to the full article. Use this template:

```
<template>
  <div class="article-preview">
      <h1 v-text="article.title" />
      <p v-text="article.description" />
      <span>Read more...</span>
  </div>
</template>
```

☐ 10. Write the export statement for `ArticlePreview`.

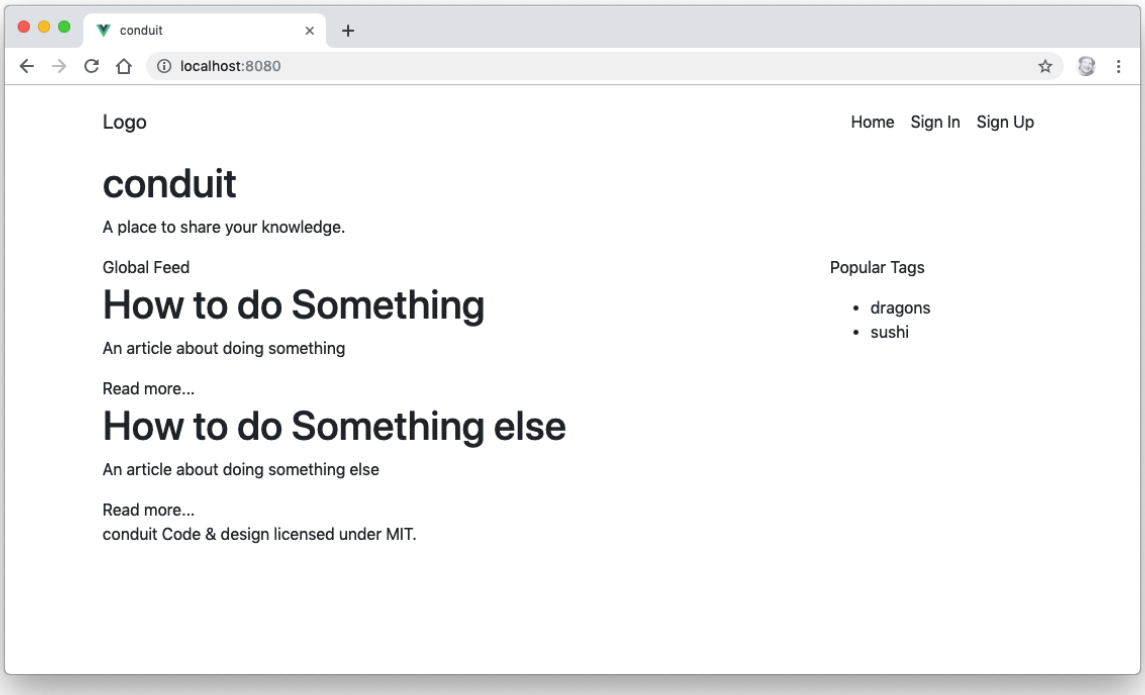
```
<script>
export default {
    name: 'ArticlePreview',
}
</script>
```

☐ 11. `ArticlePreview` receives a prop, article, from its parent component. Add this prop to configuration object.

```
<script>
export default {
    name: 'ArticlePreview',
    props: {
    article: { type: Object, required: true }
    },
}
```

```
</script>
```

- [ ] 12. Make sure that all of your components have export statements and that the exports have the right names.
- [ ] 13. Import the `GlobalFeed` component into the `Home` component, and replace the Home Content Placeholder in its template with `<GlobalFeed />`.
- [ ] 14. Run `npm run serve` and preview your application in your browser. It should look like this:



Lab 12: Styling Vue Components

In this lab, you'll learn to style components using Vue.js's scoped CSS.

- ☐ 1. If it's not already running, start up your development server by running npm run serve. Open your browser and go to http://localhost:8080.
- ☐ 2. Is VS Code, open the Home componenent.
- ☐ 3. Create a scoped <style> element under the <script> element.
- ☐ 4. Create the following css selectors inside <style scoped>

```
.banner {}
.banner p {}
.banner h1 {}
.feed-toggle {}
.sidebar {}
.sidebar p{}
```

- ☐ 5. Add a background color to the .banner with a value of #5cb85c

```
.banner {
  background:#5cb85c;
}
```

- ☐ 6. Center the paragraph text, make it larger and white, and set the font-weight to a lighter value.

```
.banner p {
  color: #fff;
  text-align: center;
  font-size: 1.5rem;
  font-weight: 300;
}
```

- ☐ 7. Make the h1 in the banner bolder, centered and larger.

```
.banner h1 {
  font-weight: 700;
  text-align: center;
  font-size: 3.5rem;
}
```

- ☐ 8. Give the sidebar a background color, padding, and rounded corners.

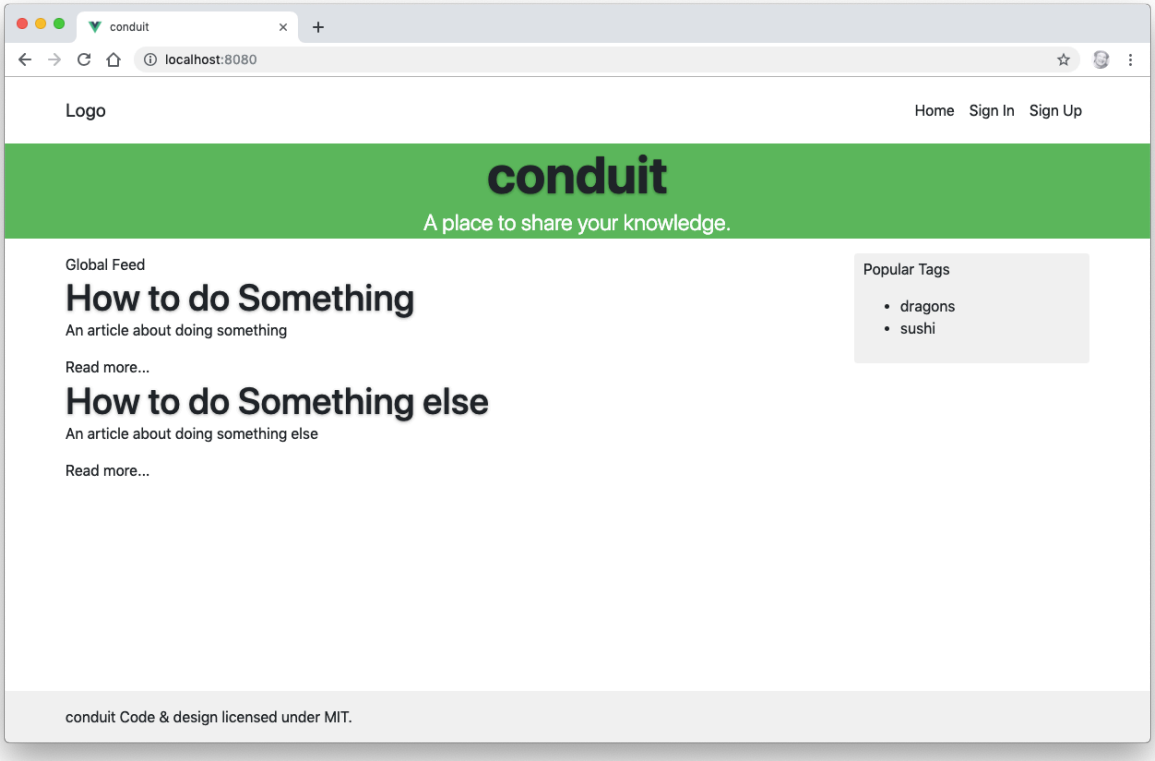
```
.sidebar {
  padding: 5px 10px 10px;
  background: #f3f3f3;
```

```
    border-radius: 4px;
  }
```

- [ ] 9. Open the Footer component.
- [ ] 10. Add a scoped style block to it.
- [ ] 11. Position and style the footer with the following CSS:

```
footer {
  background: #f3f3f3;
  margin-top: 3rem;
  padding: 1rem 0;
  position: absolute;
  bottom: 0;
  width: 100%;
}
```

- [ ] 12. When it's done, it will look like this:



Lab 13: Computed Properties

In this lab, you'll use computed properties to add links to each article's preview.

☐ 1. Open the ArticlePreview component in your code editor.

☐ 2. Create the computed property in the exported Vue instance object.

```
export default {
  name: 'ArticlePreview',
  props: {
    article: { type: Object, required: true }
  },
  computed: {
  }
}
```

☐ 3. Add a method to the computed property called articleLink(), which returns the "slug" (or path) to the current article.

```
...
computed: {
  articleLink() {
    return {
      slug: this.article.slug
    }
  }
}
```

☐ 4. Use the v-bind directive in the template in ArticlePreview to link the "Read more..." text to the article slug.

Lab 14: Methods and State

In this lab, you'll create a "favorite" button associated with each article preview, which can be toggled on and off. The state of the "favorite" button will be tracked using the data property of the Vue instance, and it will be changed using a method.

- ☐ 1. Add a 'data' property to the ArticlePreview component and create a Boolean property called **`favorited`** in it. Remember that data must be a function when you define it inside a component.
- ☐ 2. Create a method inside ArticlePreview named toggleFavorite() that checks the value of the **`favorited`** property and changes it to the opposite of its current value.
- ☐ 3. Make a button inside the ArticlePreview template that calls the toggleFavorite() method when its clicked and updates its color or text or icon based on the value of **`favorited`**.
- ☐ 4. Challenge: put a value on the like button that tracks how many times the button for an article has been clicked.

Lab 15: Events

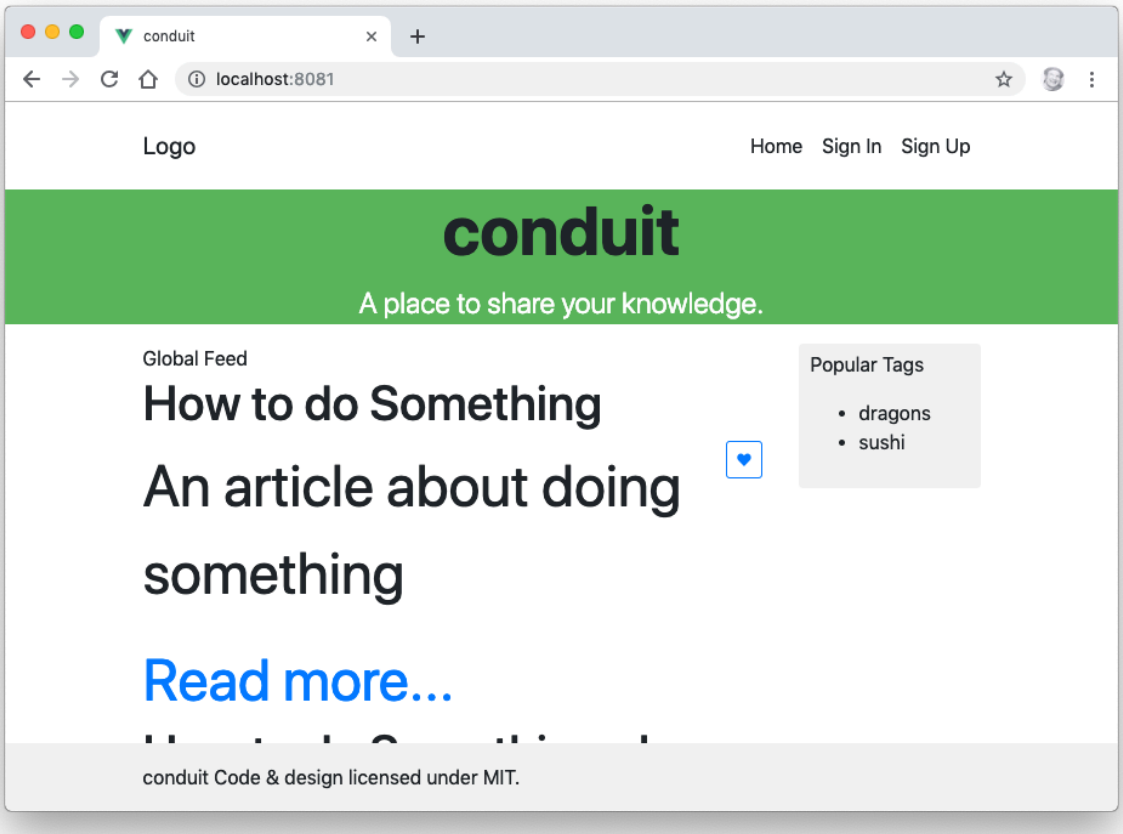
In this lab, you'll add a font-size control to the ArticleList component that will control the font size of all the article preview instances.

☐ 1. Add a data property to the ArticleList component called articleFontSize with a value of 1.

☐ 2. Pass the value of articleFontSize into the ArticlePreview component by binding the style attribute:

```
<ArticlePreview
      :style="{ fontSize: articleFontSize + 'em' }"
      …
/>
```

☐ 3. Start the development server (using npm run serve) and try changing the value of articleFontSize to 3. You should see the article description text get larger (you may need to refresh your browser).



☐ 4. Set articleFontsSize back to 1.

☐ 5. Create a button inside the articlePreview component labeled "Enlarge Text" and make it emit a custom "enlarge-text" event on click.

```
<button v-on:click="$emit('enlarge-text',0.1)">
  Enlarge Text
</button>
```

☐ 6. Listen for the “enlarge-text” event in the ArticleList component and increase the value of articleFontSize by 0.1 when it’s received.

```
<ArticlePreview
:style="{ fontSize: articleFontSize + 'em' }"
v-on:enlarge-text="articleFontSize += 0.1"
/>
```

☐ 7. Add another button named “Shrink Text” and make it work.

☐ 8. Challenge: Make clicking the button only affect the article description, not the text of the button itself.

Lab 16: Component Lifecycle

In this lab, you'll create a method to load the articles using axios when the component is mounted.

☐ 1. Create a folder inside src named common and create a file in it named config.js

☐ 2. Inside config.js, export a const named API_URL that will contain the url of the public API we'll be replacing our articles.json file with.

```
export const API_URL =
"https://conduit.productionready.io/api";
```

☐ 3. Export API_URL as the default export for config.js.

```
export default API_URL;
```

☐ 4. install axios and vue-axios

```
> npm install --save axios vue-axios
```

☐ 5. Create a file in src/common named api.service.js and import Vue, axios, VueAxios and API_URL into it

```
import Vue from "vue";
import axios from "axios";
import VueAxios from "vue-axios";
import { API_URL } from "@/common/config";
```

☐ 6. Create a const named ApiService in api.service.js, which will hold an object.

```
const ApiService = {
};
```

☐ 7. Make a method named init() in ApiService. Inside it, set up VueAxios and axios.

```
const ApiService = {
  init(){
    Vue.use(VueAxios,axios);
    Vue.axios.defaults.baseURL = API_URL;
  }
};
```

☐ 8. Export ApiService.

```
export default ApiService;
```

☐ 9. Import ApiService into main.js

```
import ApiService from "./common/api.service";
```

☐ 10. Run the ApiService.init() method inside main.js

```
ApiService.init();
```

☐ 11. Create a method named fetchArticles in ArticleList.vue

```
methods: {
  fetchArticles() {
      console.log("fetching articles");
  }
}
```

☐ 12. Call `fetchArticles()` in the `mounted()` lifecycle hook.

```
mounted() {
  this.fetchArticles();
}
```

☐ 13. Make sure the development server is running and refresh your page with the JavaScript console open. The fetchArticles() method should log its message once the page load.

☐ 14. Make a new method called query in the ApiService object in api.service.js

```
  query(resource, params) {
    return Vue.axios.get(resource, params).catch(error => {
      throw new Error(`[RWV] ApiService ${error}`);
    });
  },
```

☐ 15. Import ApiService into ArticleList.vue

```
import ApiService from '../common/api.service.js';
```

☐ 16. Modify fetchArticles to use the query method.

```
fetchArticles() {
      console.log("fetching articles");
      return ApiService.query("articles").catch(error => {
      throw new Error(`[RWV] ApiService ${error}`);
      });
    }
```

☐ 17. Change ArticleList's mounted method to an async function and set the value of this.articles inside it to the returned value from the fetchArticles() method.

```
  async mounted() {
    let results = await this.fetchArticles();
    this.articles = results.data.articles;
    console.log(this.articles);
  },
```

☐ 18. Remove the articles.json import from ArticleList.vue and set articles to an empty object in the data property.

```
data(){
  return{
    articleFontSize: 1,
    articles:{}
  }
}
```

- [ ] 19. Challenge: Fix the footer so that it sticks to the bottom of the list of articles when it's populated.

Lab 17: Forms

In this lab, you'll implement an input box that will filter the currently displayed list of articles based on the words entered into it.

☐ 1. Create an input in the ArticleList component.

```
<input
        class="form-control"
        v-model.number="searchDetails"
        placeholder="filter articles"
        >
```

☐ 2. Make a new computed property in ArticleList called filterIt:

```
computed: {
  filterIt: function(){
    var self = this;
    return this.articles.filter(function(a){
      return a.title.indexOf(self.searchDetails) > - 1
    }
    )
  },
},
```

☐ 3. Add the searchDetails property to the data object and set it to a blank string.

```
data(){
      return{
          articles: {},
          articleFontSize: 1,
          searchDetails: ''
      }
},
```

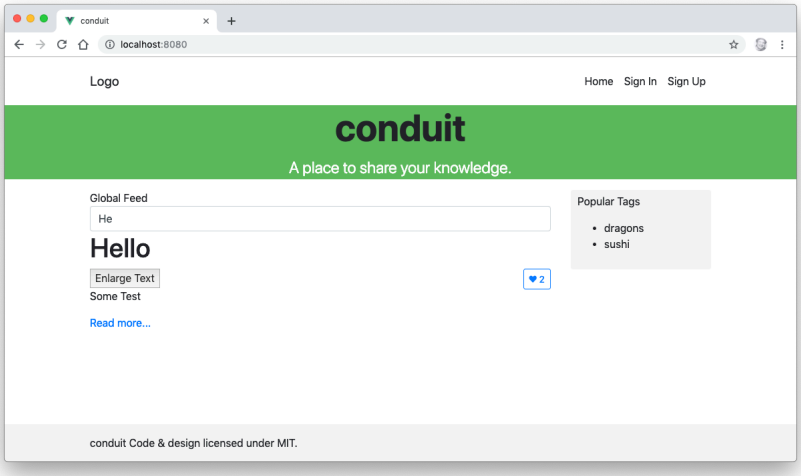
☐ 4. Modify the v-for directive in ArticleList.vue to use FilterIt instead of articles:

```
v-for="(article, index) in filterIt"
```

☐ 5. Start up your development server and try typing letters into the search form to see how the list is filtered.

☐ 6. Challenge: Make the search case-insensitive.

☐ 7. Challenge: Make the search consider the description text as well as the title.


conduit
localhost:8080
Logo
Home
Sign In
Sign Up
conduit
A place to share your knowledge.
Global Feed
He
Hello
Enlarge Text
2
Some Test
Read more...
Popular Tags
dragons
sushi
conduit Code & design licensed under MIT.

Lab 18: Slots

Lab 19: Mixins

Lab 20: Implementing Vuex

In this lab, you'll convert the Conduit application to use a centralized store with Vuex.

☐ 1. Install Vuex

```
npm install vuex --save
```

☐ 2. Make a new directory in src named store

☐ 3. Make a new file, index.js inside store

☐ 4. Make a new file, home.module.js inside store

☐ 5. In store/index.js, import Vue, Vuex, and home.module.js

```
import Vue from "vue";
import Vuex from "vuex";
import home from "./home.module";
```

☐ 6. Install Vue in store/index.js with Vue.use

```
Vue.use(Vuex);
```

☐ 7. Export the store in store/index.js:

```
export default new Vuex.Store({
  modules: {
    home,
  }
});
```

☐ 8. Import store into main.js

```
import store from "./store";
```

☐ 9. Pass the store into the root Vue instance.

```
new Vue({
  store,
  render: h=>h(App)
}).$mount("#app");
```

☐ 10. Make a file named actions.type.js in the store directory and export a FETCH_ARTICLES action

```
export const FETCH_ARTICLES = "fetchArticles";
```

☐ 11. Import FETCH_ARTICLES into home.module.js

```
import { FETCH_ARTICLES } from "./actions.type";
```

☐ 12. Make a new file named mutations.type.js in /store with the following content:

```
export const FETCH_END = "setArticles";
export const FETCH_START = "setLoading";
export const SET_TAGS = "setTags";
```

```
export const UPDATE_ARTICLE_IN_LIST =
"updateArticleInList";
```

☐ 13. Import mutations into home.module.js

```
import {
  FETCH_START,
  FETCH_END,
  SET_TAGS,
  UPDATE_ARTICLE_IN_LIST
} from "./mutations.type";
```

☐ 14. Make a new API service in `api.service.js` called `ArticlesService`.

```
export const ArticlesService = {
  query(type, params) {
    return ApiService.query("articles" + (type === "feed" ?
"/feed" : ""), {
      params: params
    });
  }
};
```

☐ 15. Import ArticleService into home.modules.js

```
import { ArticlesService } from "@/common/api.service";
```

☐ 16. Create a state object in home.modules.js

```
const state = {
  tags: [],
  articles: [],
  isLoading: true,
  articlesCount: 0
};
```

☐ 17. Create a getters object in home.modules.js

```
const getters = {
  articlesCount(state) {
    return state.articlesCount;
  },
  articles(state) {
    return state.articles;
  },
```

```
  isLoading(state) {
    return state.isLoading;
  },
  tags(state) {
    return state.tags;
  }
};
```

☐ 18. Make an actions object in home.module.js:

```
const actions = {
  [FETCH_ARTICLES]({ commit }, params) {
    commit(FETCH_START);
    return ArticlesService.query(params.type,
params.filters)
      .then(({ data }) => {
        commit(FETCH_END, data);
      })
      .catch(error => {
        throw new Error(error);
      });
  },
};
```

☐ 19. Create a mutations object:

```
const mutations = {
  [FETCH_START](state) {
    state.isLoading = true;
  },
  [FETCH_END](state, { articles, articlesCount }) {
    state.articles = articles;
    state.articlesCount = articlesCount;
    state.isLoading = false;
  },
  [SET_TAGS](state, tags) {
    state.tags = tags;
  },
  [UPDATE_ARTICLE_IN_LIST](state, data) {
```

```
      state.articles = state.articles.map(article => {
        if (article.slug !== data.slug) {
          return article;
        }

        article.favorited = data.favorited;
        article.favoritesCount = data.favoritesCount;
        return article;
      });
    }
  };
```

☐ 20. Export everything!

```
export default {
  state,
  getters,
  actions,
  mutations
};
```

☐ 21. Import mapGetters and FETCH__ARTICLES into ArticleList.vue

```
import { mapGetters } from "vuex";
import { FETCH_ARTICLES } from "../store/actions.type";
```

☐ 22. Remove articles from the data property in ArticleList.vue

☐ 23. Modify the mounted lifecycle hook to simply call fetchArticles(). Note: It no longer needs to be an async function.

```
mounted() {
  this.fetchArticles();
},
```

☐ 24. Modify the fetchArticles method to dispatch the FETCH_ARTICLES action.

```
    fetchArticles() {
      this.$store.dispatch(FETCH_ARTICLES,this.listConfig);
    },
```

☐ 25. Create the listConfig computed property in ArticleList

```
    listConfig() {
      const { type } = this;
```

```
      return {
        type,
      };
    },
```

☐ 26. Use mapgetters to map articles and the other getters to local computed properties by adding the following to the computed property in ArticleList.

```
    ...mapGetters(["articlesCount", "isLoading",
"articles"])
```

☐ 27. Add a v-if directive to the ArticleList component to hide the list until the AJAX call is finished.

```
<template>
  <div>
    <div v-if="isLoading" class="article-preview">Loading
articles...</div>
    <div v-else>
      <div v-if="articles.length === 0" class="article-
preview">
        No articles are here... yet.
      </div>
      <ArticlePreview
        v-on:enlarge-text="articleFontSize += 0.1"
        :style="{ fontSize: articleFontSize + 'em' }"
        v-for="(article, index) in articles"
        :article="article"
        :key="article.title + index"
      />
    </div>
    </div>
</template>
```

☐ 28. Start the development server and view the app in a browser.

☐ 29. Challenge: Display tags along with each article preview.

☐ 30. Challenge: Populate the Popular Tags box with a list of tags from the currently displayed article previews.

☐ 31. Challenge: Clicking on a tag in the list of popular tags should filter the display of articles.

Lab 21: Routing

So far, our application only has one screen. In this lab, you'll start to implement additional routes. Routing in modern JavaScript frameworks works through the JavaScript reacting to changes in the browser address to load or show different views. In actuality, everything is happening in the same HTML page, but the data and view are changing. This is what we mean when we refer to a Single Page Application (or SPA).

In this lab, you'll install and configure Vue Router, and then you'll create the first two necessary routes for this application: Sign In and Sign Up.

☐ 1. Install vue-router.

```
npm install --save vue-router
```

☐ 2. Make a new directory in src named router.

☐ 3. Create a file named index.js inside src/router

☐ 4. import Vue and Router into src/router/index.js and install Router using Vue.use

```
import Vue from "vue";
import Router from "vue-router";

Vue.use(Router);
```

☐ 5. Make and export a new Router component and pass it a configuration object containing a routes array:

```
export default new Router({
      routes: []
});
```

☐ 6. Make the first (default) route:

```
routes: [{
      name: "home",
      path: "/",
      component: ()=>import("@/components/Home"),
}]
```

☐ 7. Import the router module into main.js

```
import router from "./router";
```

☐ 8. Pass router into the root instance:

```
new Vue({
  router,
  store,
  render: h => h(App)
```

```
}).$mount("#app");
```

☐ 9. Open App.vue and replace the <Home /> component with <router-view>:

```
<template>
  <div id="app">
    <Header />
    <router-view></router-view>
    <Footer />
  </div>
</template>
```

☐ 10. Remove the import for Home and remove Home from the components property.

☐ 11. Open the development server and open the application in your browser. It should work the same, but you'll notice that "/#/" has been added to the URL.

☐ 12. Now that we have a router, we can define additional routes and display different when links or buttons are clicked.

☐ 13. Open the Header component for editing.

☐ 14. Find the Sign In link, and replace it with a router-link component:

```
<router-link
  class="nav-link"
  active-class="active"
  exact
  :to="{ name: 'login' }"
  >
  Sign in
</router-link>
```

☐ 15. View the application in your browser and click on the Sign In link. It won't work, because the login route doesn't exist yet.

☐ 16. Create a new route, in router/index.js, with the name login, the path "/login", and that will use the "login" component.

```
routes: [
        {
            name: "home",
            path: "/",
            component: ()=>import("@/components/Home"),
        },
        {
            name:"login",
            path: "/login",
```

```
            component: () => import("@/components/Login")
        }
    ]
```

☐ 17. Make a new component named Login and put a login form in the <template>:

```
<template>
  <div>
    <h1>Log In Here</h1>
    <form v-on:submit.prevent="onSubmit();">
        <fieldset class="form-group">
            <input
            class="form-control form-control-lg"
            type="text"
            placeholder="Email"
            />
        </fieldset>
        <fieldset class="form-group">
            <input
            class="form-control form-control-lg"
            type="password"
            placeholder="Password"
            />
        </fieldset>
        <button
           class="btn btn-lg btn-primary pull-xs-right">
           Sign in
        </button>
    </form>
  </div>
</template>
```

☐ 18. Use <router-link> to link the Logo to the home page.

```
<router-link
  class="navbar-brand"
  :to="{ name: 'home' }">Logo
</router-link>
```

☐ 19. Link the Home navigation link to the home page.

```
<router-link
  class="nav-link"
  active-class="active"
  exact
  :to="{ name: 'home' }">
Home</router-link>
```

☐ 20. Challenge: Make the Sign Up page and routes.

☐ 21. Challenge: Use a <router-view> inside the Home.vue component to render a differently list (or differently-styled list) of articles when the user clicks a link (maybe to the right of the Global Feed title). HINT: add a children property to the home route and make it an array of route objects.

Lab 22: AJAX

In this lab, you'll implement sign up functionality along with login and checking of authorization.

☐ 1. Add new actions to actions.type.js for login, logout, register, update_user, and check_auth:

```
export const CHECK_AUTH = "checkAuth";
export const LOGIN = "login";
export const LOGOUT = "logout";
export const REGISTER = "register";
export const UPDATE_USER = "updateUser";
```

☐ 2. Copy the auth.module.js file from labs/lab22/store into your project.

☐ 3. Create the following new mutations in mutations.type.js

```
export const PURGE_AUTH = "logOut";
export const SET_AUTH = "setUser";
export const SET_ERROR = "setError";
```

☐ 4. Replace your common/api.service.js with api.service.js from labs/lab22/common/

☐ 5. Import JwtService into api.service.js

```
import JwtService from "@/common/jwt.service";
```

☐ 6. Import auth into store/index.js

```
import auth from "./auth.module";
```

☐ 7. Inject auth into the store.

```
export default new Vuex.Store({
    modules: {
      auth,
      home,
    }
  });
```

☐ 8. Copy Login.vue and Register.vue from labs/lab22/components and overwrite your existing Login and Register components

☐ 9. Test it out by registering and then by logging in.

☐ 10. Import mapGetters into Home.vue

```
import { mapGetters } from "vuex";
```

☐ 11. Use mapGetters to merge the isAuthenticated computed property into the Home.vue component

```
computed: {
```

```
        ...mapGetters(["isAuthenticated"]),
    }
```

☐ 12. Use isAuthenticated in Home.vue to conditionally show a logged-in user message in the sidebar.

```
<h1 v-if="isAuthenticated">Welcome authenticated user</h1>
```

☐ 13. Challenge: display a logout button instead of the Login and signup buttons when a user is logged in.

Lab 23: Testing with Jest

If you run the unit tests in your project now, you'll see that the tests on Home.vue fail. In this lab, you'll learn about the jest testing utilities and properly configure your tests to use Router.

- ☐ 1. Visit https://vue-test-utils.vuejs.org/guides/#getting-started and read about using the vue test utilities with Vue Router and Vuex.
- ☐ 2. Modify Home.spec.js as necessary so that the test in it passes.

Lab 24: Transitions and Animations